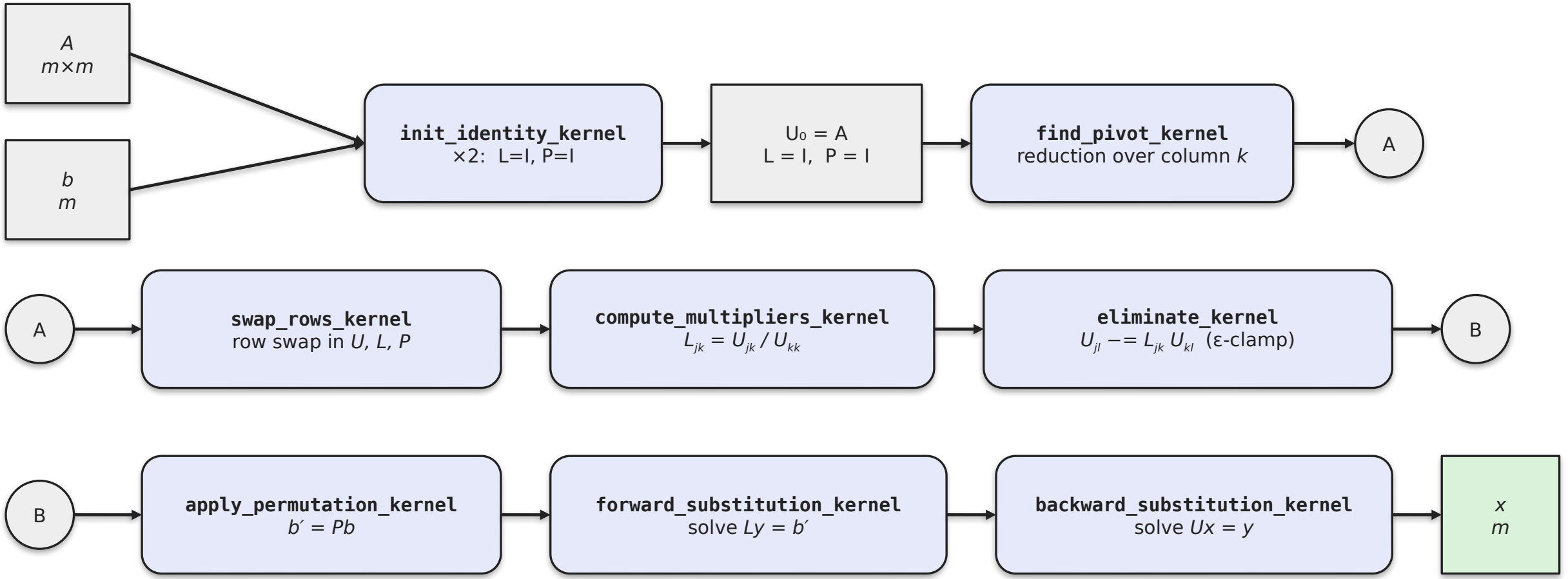
LU Solver on the GPU

From FP64 kernels to tensor cores, watched by NixNan

Ganesh Gopalakrishnan
SC26 tutorial

Algorithm: LU with partial pivoting, solve $Ax = b$

$PA = LU$ pivot: $p = argmax_{i \geq k} |U_{ik}|$ multiplier: $L_{jk} = U_{jk} / U_{kk}$
eliminate: $U_{jl} -= L_{jk} U_{kl}$ $(j>k, l \geq k)$ forward: $Ly = Pb$ backward: $Ux = y$



NixNan knobs: environment variables

INSTR_BEGIN / END	instr. index window	**NIXNAN_TIMEOUT**	self-terminate, secs
TOOL_VERBOSE	setup diagnostics	**LOGFILE**	redirect output
ENABLE_FUN_DETAIL	name on every rerun	**LINE_INFO**	source file:line (YMMV)
PRINT_ILL_INSTR	print culprit SASS	**LOG_KERNELS**	log launches only
SAMPLING	every N-th launch	**MAX_ERRORS**	stop after N reports
INSTRUMENT_EXCEPTIONS	master switch	**TIME_KERNELS**	per-kernel time
INSTR_MEM	also check STG (excn → mem)	**FUNCTION_WHITELIST**	only these kernels
HISTOGRAM	exponent histogram	**FUNCTION_BLACKLIST**	all but these
BIN_SPEC_FILE	ranges to watch		

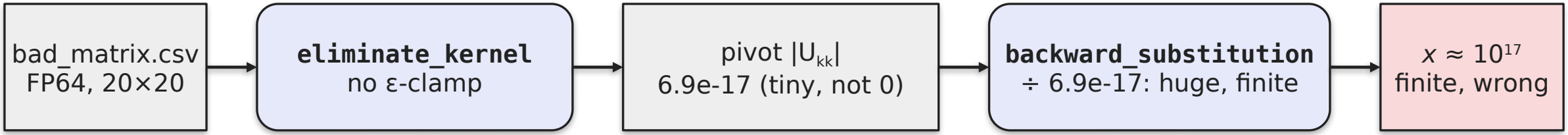
For example, ksel-autonixnan scripts (using the above env vars) the collection of kernel names and execution times, so users can then white- or black-list exactly what to observe in detail.

TIME_KERNELS: the first-launch cost

Kernel	Launches	First launch (µs)	Median later (µs)	Ratio
`init_identity_kernel`	2	312,451	21	≈ 14,900×
`find_pivot_kernel`	19	298,100	20	≈ 14,900×
`swap_rows_kernel`	19	290,205	25	≈ 11,600×
`compute_multipliers_kernel`	19	295,432	20	≈ 14,800×
`eliminate_kernel`	19	278,808	20	≈ 13,900×
`apply_permutation_kernel`	1	300,320	—	—
`forward_substitution_kernel`	1	326,736	—	—
`backward_substitution_kernel`	1	324,597	—	—

- First launch of every kernel: ≈280–330 ms. Later launches: ≈20–25 µs, four to five orders of magnitude less.
- The timer starts before NVBit rewrites the kernel’s SASS (once per function) and stops after, so the first number is instrumentation cost, not kernel time.
- Kernels launched once (last three rows) only ever show this one-time cost.
- Library-heavy binaries pay far more: the cuSOLVER-based pytorch_slogdet run was killed at 120 s during instrumentation.

Before the clamp: a silent failure



	matrix.csv	bad_matrix.csv
Smallest pivot $\lvert U_{kk}\rvert$ (read back from the GPU)	1.445	6.9e-17
Residual $\lVert Ax - b\rVert_2$	9.7e-16	33.9
Solution x	≈ 1, finite	up to 1.7e17, finite
nixnan exceptions (all precisions, memory)	none	none

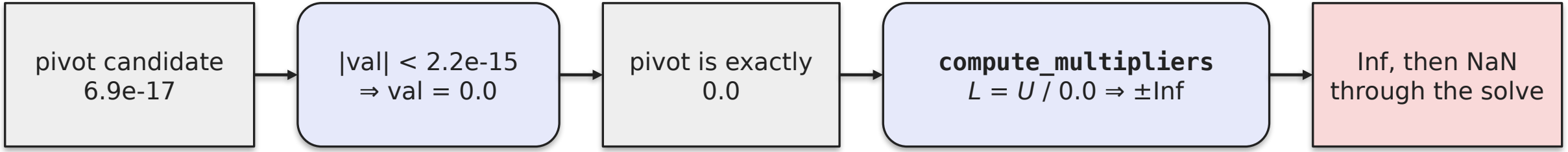
The program exits 0 and prints numbers. Only the residual shows that something is wrong. nixnan has nothing to report: no instruction produced a NaN, an Inf or a division by zero.

FP64, no clamp, plain LD_PRELOAD run; logs/clamp_story/fp64_clampoff_*

The defensive fix: snap tiny values to zero

```
val = U[j][l] - L[j][k] * U[k][l];
if (fabs(val) < 10.0 * DBL_EPSILON)   // 2.2e-15
    val = 0.0;                        // set to zero if close to machine epsilon
U[j][l] = val;
```

in eliminate_kernel (lu_solve_gpu.cu, lines 210–211)



- Intent: remove round-off noise, keep zeros exact, make a singular matrix visible.
- Effect here: “inaccurate” (residual 33.9) becomes “divide by zero”, and the failure is now loud.
- The clamp’s threshold is absolute (2.2e-15), whatever the scale of the matrix.

Stage 1 results: well-conditioned vs ill-conditioned input

	matrix.csv	bad_matrix.csv
Residual $\|Ax - b\|_2$ (uninstrumented)	9.68e-16	nan
Binade-threshold reports	99	63
FP64 NaN (repeats)	0 (0)	316 (2171)
FP64 Infinity (repeats)	0 (0)	4 (0)
FP64 Division-by-zero (repeats)	0 (0)	4 (4)

matrix.csv
clean solution, no exceptions

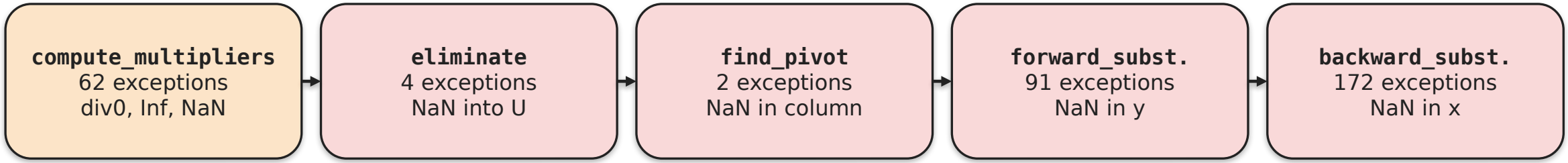
bad_matrix.csv
near-zero pivot → division by zero → NaN
through the whole solve

Following the first exception back to the clamp

PRINT_ILL_INSTR=1 ENABLE_FUN_DETAIL=1 (FP64, clamp on, bad_matrix.csv); the first report:

```
error [div0] detected in operand 1 of instruction MUFU.RCP64H R3, R5
in function compute_multipliers_kernel at lu_solve_gpu_clamp_study.cu:216  of type f64
```

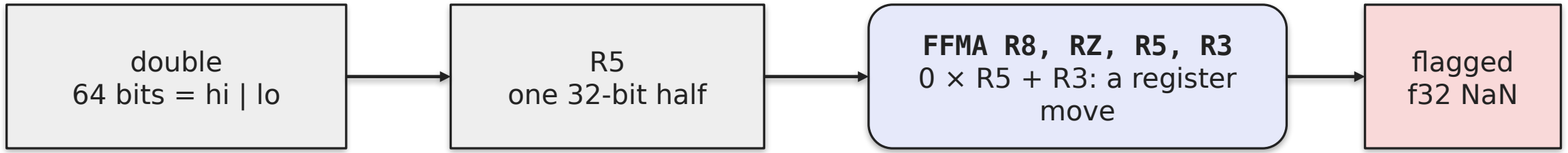
Line 216 is the division $L_{jk} = U_{jk} / U_{kk}$: the reciprocal of the zeroed pivot is the first thing that goes wrong.



- The first report names the kernel, the instruction and the source line; the rest is propagation.
- The cause is upstream of it: the clamp wrote 0.0 into the pivot. nixnan shows where the zero is divided by, not why it is zero.
- Counts are per kernel, from 331 detail lines (run_clamp_story.sh step2).

Binary instrumentation reveals exceptions you did not expect

- The source computes only in double, yet bad_matrix.csv also reports 6 FP32 NaN detections (93 repeats).
- Cause: ptxas uses the idiom FFMA R8, RZ, R5, R3 (0 × R5 + R3 = R3) as a register move, shuffling the 32-bit halves of a double. nixnan sees an f32 FFMA whose operand R5 decodes as NaN.
- Lesson: a SASS-level detector reports what the compiled code does, not what the source says. A nonzero count in a format the source never uses is not necessarily that format's arithmetic.



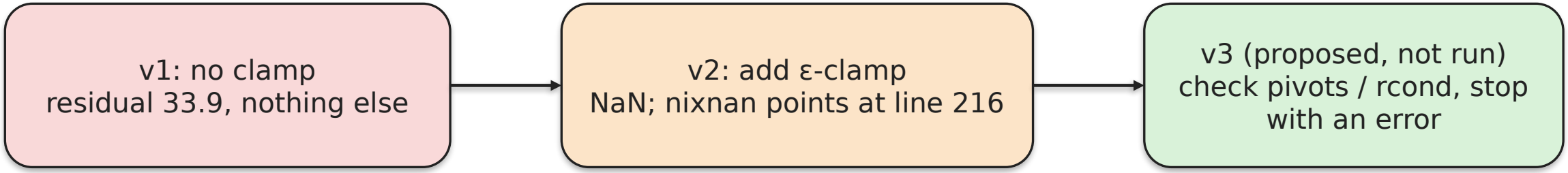
The same two steps in FP32, and at 128×128

bad matrix	no clamp: pivot	residual	nixnan	clamp: pivot	residual	clamp: nixnan
FP64 20×20	6.9e-17	33.9	none	0.0	NaN	NaN 316, Inf 4, Div0 4
FP32 20×20	3.0e-8	26.2	none	0.0	NaN	NaN 253, Inf 8, Div0 2
FP64 128×128	1.1e-15	219	none	0.0	NaN	NaN 142, Inf 19, −Inf 2, Div0 2
FP32 128×128	1.0e-6	92.8	none	0.0	NaN	NaN 107, Inf 21, −Inf 2, Div0 1

- Without the clamp: finite wrong answers (residual 26–219) and nothing for nixnan to report, in both precisions and both sizes. With it: exact zero pivot, NaN, and nixnan counts the NaN, Inf and division by zero.
- The well-conditioned matrices stay clean in every configuration (residual 1e-15 in FP64, 5e-7 in FP32; no exceptions).
- 128×128 matrix: product of two perturbed tridiagonal bands, cond ≈ 3×10^{17} (80-digit arithmetic; gen_matrix_n.py); bad_matrix.csv: ≈ 2×10^{17}.
- FP32 solves a rounded copy of the matrix. Its pivot at 128×128 (1.03e-6) sits just under the FP32 clamp threshold (1.19e-6): one seed, no claim about other matrices.

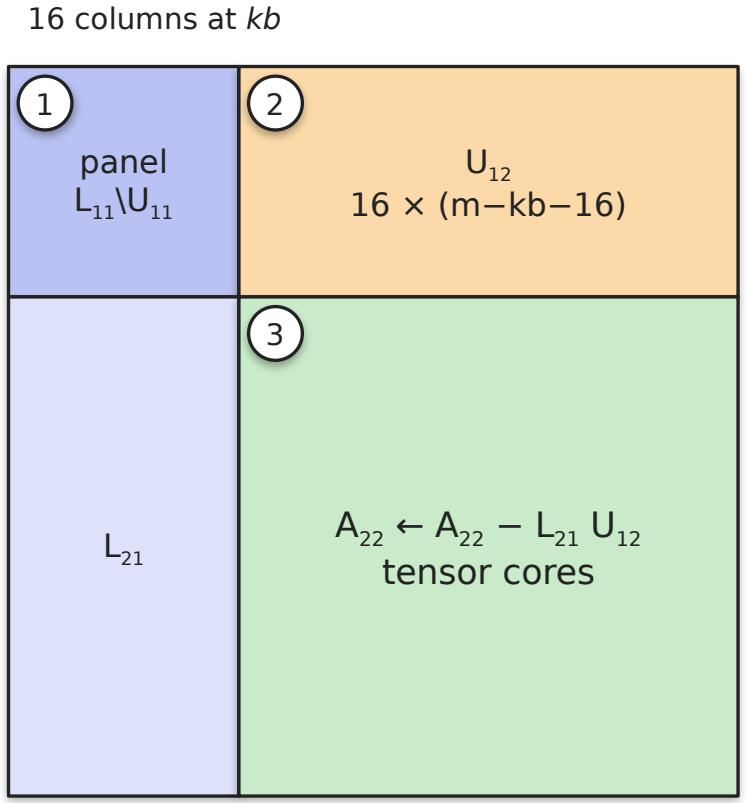
What each check can see: nixnan, the residual, a pivot check

	v1: no clamp	v2: clamp
nixnan (NaN / Inf / div-by-zero)	blind	finds it: kernel, instruction, line
Residual $\|Ax - b\|$ (one matrix-vector product)	33.9: finds it	NaN: finds it
Pivot / condition check before dividing	finds it, before the solve	finds it, before the solve



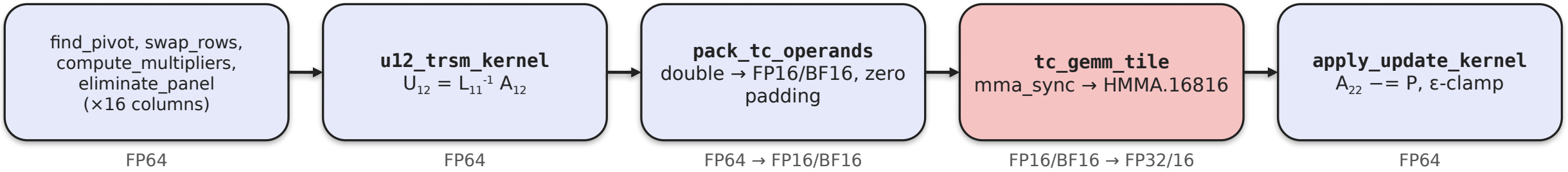
- nixnan finds exceptions, not ill-conditioning: v1 is invisible to it. A residual or pivot check covers that gap.
- The clamp did not fix the problem; it moved it from “silently wrong” to “divide by 0.0”.
- A sound check is relative (pivot against the matrix norm) and reports an error, as LAPACK’s info and rcond do.

LU Adaptation to Tensor Cores Involves Panel Factorization

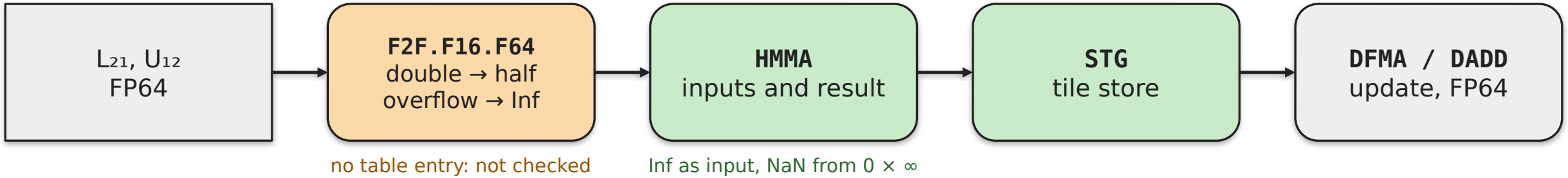


1. panel, FP64: pivot search, row swap, multipliers, elimination on the 16 panel columns
2. $U_{12} = L_{11}^{-1}\,A_{12}$, FP64 triangular solve
3. $A_{22} -= L_{21}\,U_{12}$: round to FP16/BF16, WMMA on tensor cores, subtract in FP64

Repeat for kb = 0, 16, 32, …; a last block narrower than 16 columns is panel-only.



Tensor-core LU: where exceptions arise, and what we saw

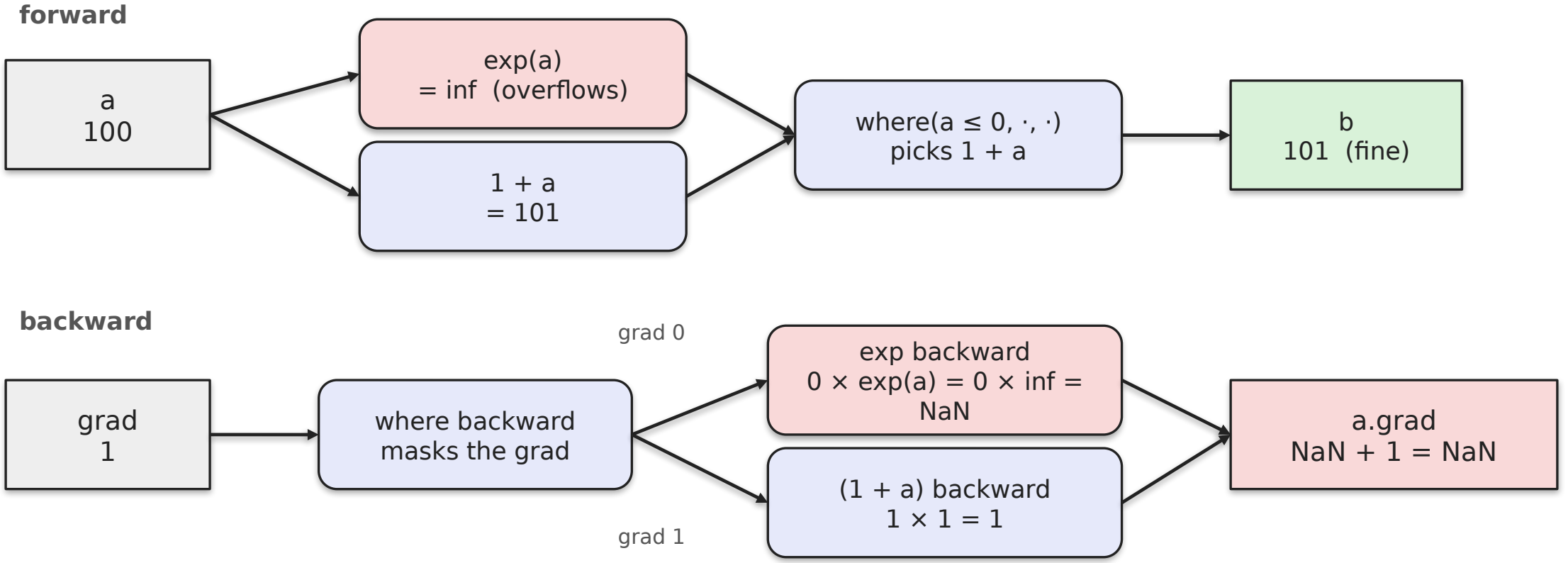


Not all instructions are documented upfront, so we had to discover, ad hoc and whenever possible, which ones NixNan actually checks.

Input	fp16acc32	fp16acc16	bf16acc32	FP64 original
`matrix.csv residual`	1.1e-4	2.0e-4	4.4e-4	9.7e-16
`bad_matrix.csv`	NaN	NaN	NaN	NaN
`rand:64:100000`	NaN	NaN	0.17	–

torch.where: a well-defined forward, a NaN gradient

```
b = torch.where(a <= 0, torch.exp(a), 1 + a),   a = 100 (float32, on GPU)
```



NixNan on the GPU kernels: inf in an FMUL of the exp kernel; then NaN in an FMUL (0 × inf) and its STG; then NaN in an FFMA (accumulation) and its STG.
Report: FP32 NaN at 5 sites, Infinity at 2 sites, FP32 memory NaN at 3 sites; all of it inside PyTorch's elementwise kernels, none in user code.

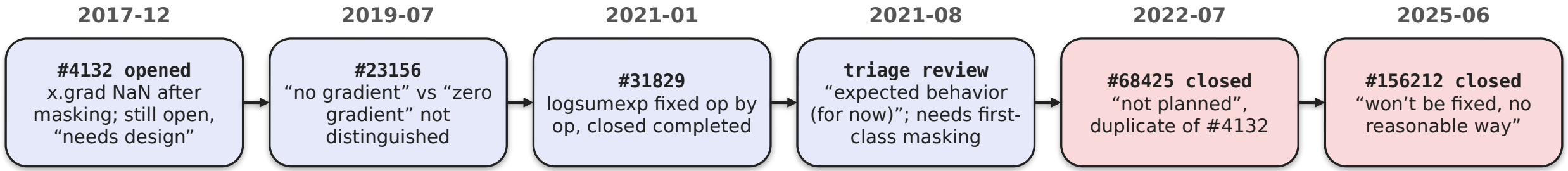
torch.where: why it stays, and the workaround

- Issue #68425: closed “not planned” (2022-07-25), as a duplicate of the still-open #4132. Reproduces unchanged on torch 2.3.1.
- Stated reason, from a maintainer: “it’ll stay like it is and nans propagate … It breaks IEEE754 simply because we don’t have the metadata available to track invalid elements.” Autograd has no “undefined grad”.
- Efficiency came up only as the cost of a fix: tracking a mask in forward “will require the CPU-GPU sync”. No maintainer says it is left for speed.
- The fix is the user’s: keep the unselected branch finite (“double where”). Thread: the same applies to log.
- Mask multiply does not help: m × exp(a) still computes 0 × inf = NaN, now in the forward value too (b = nan), so the gradient is NaN as well.

form (a = 100, float32, CUDA)	b	a.grad
`issue: where(a<=0, exp(a), 1+a)`	101	nan
`mask multiply: m × exp(a) + (1+a) × ~m`	nan	nan
`double where: exp(a.clamp_max(0))`	101	1.0
`double where: exp(where(m, a, 0))`	101	1.0

Sources: github.com/pytorch/pytorch/issues/68425; native runs, torch_where/workarounds.py.

torch.where: a decade of “won’t fix”, and what a fix costs



Why a fix is hard

- Needs per-element “unused” vs “zero” metadata (MaskedTensor-style)
- NaN is made by the op before where: fix is per op, or fused
- A forward mask m must be read on the CPU: a sync

m = (a ≤ 0): where’s own boolean condition, 1 byte per element, computed from the data on the GPU. Its values are data dependent; the sync comes only when the CPU reads it, e.g. (~m).any().item(), to decide what to do. Double where uses m to zero the unselected input; the custom Function tests the incoming gradient (g == 0) instead of m.

Measured cost of fix strategies (RTX 3090, N = 2^24, fwd+bwd)

strategy	ms / iter	vs baseline	extra memory
baseline where	1.47	1.00×	0
double where	1.99	1.35×	+16 MiB
custom Function (masked exp backward)	1.90	1.29×	+144 MiB
mask check + .item() (CPU-GPU sync)	1.57–1.69	1.07–1.15×	0

Method: 5 runs; each strategy 5 warm-up + 50 timed fwd+bwd iterations, mean ms via CUDA events; extra memory = peak allocated vs baseline. One op (exp), one GPU, no std. dev. No maintainer cites speed as the reason. Details: torch_where/FIX_COST.md.

Why the fixes need extra memory

N = 2^24 floats: one float32 tensor = 64 MiB, one bool mask = 16 MiB

retained for backward (all variants)



= 80 MiB, identical in all four (measured)

extra transient memory at the backward peak

baseline / sync check		**+0 MiB**
double where		**+16 MiB**
custom Function		**+144 MiB**

- Custom Function: its backward builds a bool g == 0 (16), a zeros tensor (64) and a full-size select (64); the baseline exp backward is just g × y (16 + 64 + 64 = 144 MiB, matches the profiler).
- Double where: a second where pass in backward reuses most freed buffers; peak rises 16 MiB (buffer not identified exactly).
- Sync check: no new tensors; it costs a stall, not memory.